

DATA 266 Lab 1: Tasks 1, 2 and 3

Character-level GPT on TinyStories, sentiment classification on Yelp Polarity, and CycleGAN
photo $\leftrightarrow$ Monet

Sivasurya Chandran · Team 6 (PairProgramming_Team_06), with Rajesh Paruchuri

github.com/Paruchuri-Rajesh/DATA_266-GPULab1

Overview

This report covers my part of all three tasks in Lab 1. In Task 1, I wrote a small GPT-style language model from scratch and trained it on characters from TinyStories. In Task 2, I compared three sentiment classifiers on the full Yelp Polarity dataset; all three learn their own word embeddings, with no pretrained vectors or language models. In Task 3, I trained a CycleGAN that turns photos into Monet-style paintings and back, and entered it in the class Kaggle competition.

Every number below is taken from the metrics files the training scripts wrote. Those files, the code, the executed notebooks and the unedited training logs are all in the team repository under `task1_llm/sivasurya_chandran/`, `task2_sentiment/sivasurya_chandran/`, `task3_gan/sivasurya_chandran/` and `reproducibility/`.

	Task 1: char-level GPT	Task 2: Yelp sentiment	Task 3: CycleGAN
Hardware	NVIDIA Tesla T4 (Google Colab), fp16	Apple M5 laptop (PyTorch MPS), fp32	NVIDIA RTX 4090 (lab PC), bf16
Data used	110K TinyStories stories (100K train / 10K val)	All 598K Yelp Polarity reviews	300 Monet paintings, 7,038 photos
Headline result	Val cross-entropy 0.639 nats/char (perplexity 1.89)	BiLSTM + attention: 95.81% test accuracy	Kaggle -45.9695 (v3 entry, team best); final model FID 101.91, MiFID 0.408 (instructor's script)
Training time	6,736 s (about 1.9 h)	About 3.2 h of compute for all three models	24,770 s (6.9 h) for the final run

Task 1: A GPT-style language model from scratch

Data and tokenisation

I sampled 110,000 stories from the TinyStories training split with a fixed seed (1337). I kept 100,000 for training and 10,000 for validation, and joined the stories in each set with newlines. That came to 89.8 million training characters and 9.0 million validation characters.

The tokeniser is just a character-to-index map, built from the sampled text. It has 109 symbols. Most are letters, digits and punctuation, but a handful are leftovers from encoding errors in TinyStories itself (characters such as $\hat{\text{A}}$, $\tilde{\text{A}}$ and €). I left those in rather than cleaning the source text, because they are rare and removing them would mean changing the data.

Training examples are 128-character windows with the target shifted one place to the right. The windows don't overlap, so one epoch is a single pass over the data: 10,964 steps at batch size 64.

Model

The model is a decoder-only Transformer, and every piece of it is in `src/model.py`. I didn't use `nn.MultiheadAttention`, `nn.Transformer` or PyTorch's fused attention function.

- Token and position embeddings, both learned and both 256-dimensional.
- Six pre-LayerNorm blocks. Each one has:
 - 8-head causal self-attention, with a lower-triangular mask so a position can only attend to itself and earlier positions;
 - a GELU feed-forward layer four times wider than the model;
 - residual connections and dropout of 0.1.
- A final LayerNorm and a linear output layer over the 109 characters.

That adds up to 4,827,648 parameters. Because the vocabulary is so small, almost all of them sit in the Transformer blocks rather than in the embedding table.

Training setup

Setting	Value	Why I chose it
Optimiser	AdamW, betas 0.9 / 0.95, weight decay 0.01 on weight matrices only	Standard for small GPTs; no decay on biases and LayerNorm
Learning rate	500-step linear warm-up to 3e-4, then cosine decay to 3e-5	Keeps early Adam updates small; the slow decay lets the loss keep improving late
Batch / context	64 sequences × 128 characters	Two or three TinyStories sentences; keeps attention cost low
Epochs	10 (109,640 steps)	Fits in about two hours on a T4
Regularisation	Dropout 0.1, gradient clipping at 1.0	Light, since the model is small relative to the data
Precision	fp16 autocast with a GradScaler	Roughly 2–3× faster on a T4

I sized the model for the budget: about 900 million training characters (ten passes over 90 million) in roughly two hours on a free Colab GPU. Pre-LayerNorm was a deliberate choice too. It trains more stably from scratch than the original post-norm layout, and I didn't want to spend GPU time hunting for a learning rate.

Results

Metric	Value
Training cross-entropy (nats/char)	0.6303
Validation cross-entropy (nats/char)	0.6385

Metric	Value
Validation perplexity	1.894
Bits per character (val)	0.921
Generalisation gap (val – train)	0.008
Top-1 next-character accuracy (val)	79.55%
Distinct-1 / 2 / 3 (sampled text)	0.252 / 0.676 / 0.886
Repeated 4-gram rate, sampled / greedy	0.006 / 0.267
Gradient norm mean / p95 / max	0.343 / 0.489 / 7.77
NaN steps / loss spikes	0 / 0
Training throughput	133,346 tokens/s
Generation throughput	217 tokens/s
Peak GPU memory	944 MB
Total training time	6,736 s

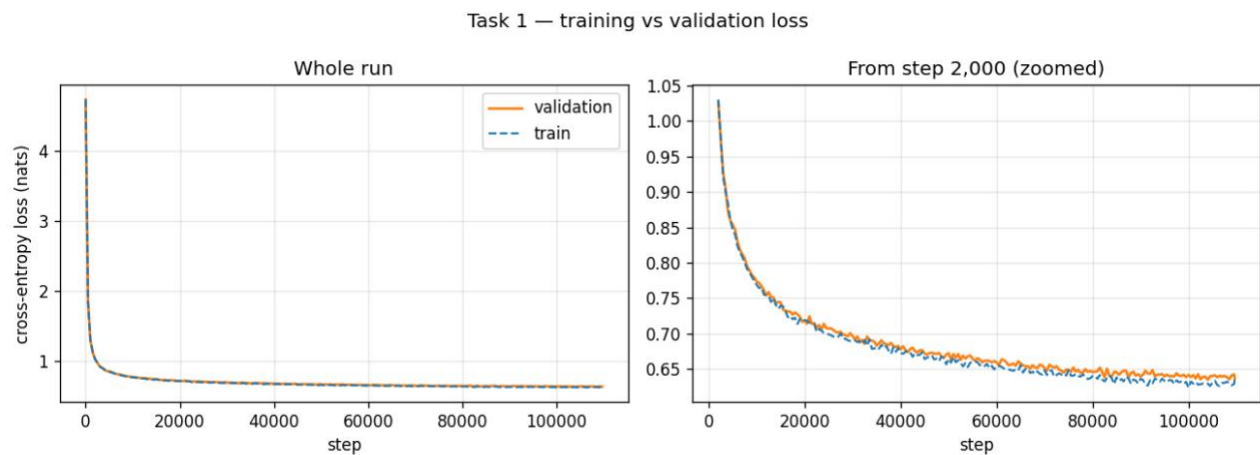


Figure 1. Training (dashed) and validation loss: the whole run on the left, zoomed in from step 2,000 on the right.

The loss drops very quickly in the first couple of thousand steps, while the model is learning basic spelling and the warm-up is still running. After that it improves slowly and steadily. The training and validation curves stay almost on top of each other for the whole run (the zoomed panel shows training just below validation), and the final gap is only 0.008 nats.

Here is the validation loss at the last check of each epoch:

Epoch	1	2	3	4	5	6	7	8	9	10
Val CE	0.772	0.714	0.686	0.674	0.669	0.656	0.647	0.638	0.640	0.637

The last three epochs are nearly flat, so the model is about as good as it will get at this size. A larger model is the more obvious next step than more epochs. Since it isn't overfitting at all, it can probably afford more capacity.

Training was stable: there were no NaN losses, and no batch losses above 1.5 times the running average. The fp16 loss scaler overflowed on 41 of the 109,640 steps (0.04%). PyTorch skips those steps automatically, so I report the gradient-norm statistics over the finite steps only.

One practical problem came up: the Colab session dropped after epoch 5 when my laptop went to sleep. The training script saves the model, optimiser, scaler and random-number state after every epoch, so I resumed from that point. My first resume attempts crashed, because the saved RNG state was being loaded onto the GPU. After I fixed that, the run retrained epoch 6 from the start and finished normally. Every start and resume is still in the raw log.

Generated text

With greedy decoding, the opening of a story usually looks like this:

Once upon a time, there was a little girl named Lily. She loved to play outside in the sunshine. One day, she went to the park with her mommy and daddy. They saw a big slide and wanted to go down it.

For the evaluation I used five prompts. Each got one greedy sample and three samples at temperature 0.8, with 300 new characters each. Locally the text is fluent: spelling is almost always right, and the sentences follow the simple TinyStories style. The problems show up over longer stretches.

Failure analysis

Repetition under greedy decoding. With the prompt "The sun was shining and", greedy decoding produced "The sun was shining and the sky was blue." over and over until it ran out of characters. That sample has a repeated 4-gram rate of 0.84. Once the phrase fills the 128-character window, the most likely next character always starts the same sentence again, so the loop never breaks. The model itself isn't the problem: the same prompt at temperature 0.8 gives varied text, and across all samples the repeated 4-gram rate is 0.006 for sampling against 0.267 for greedy. A repetition penalty or top-k sampling would be the fix to try.

Losing track of characters. In one sampled story, the main character starts as Timmy ("he"), becomes "she" a sentence later, and then turns into Jack. In the same stretch, the rock he found becomes a rocket. Each sentence is fine on its own, and "rock" to "rocket" is a very natural slip for a model that writes one character at a time. The cause is the short context: by the time the model writes the next line of dialogue, the name has often already left the 128-character window. I'd test a 256-character context and count how often the main character's name changes per story.

Grammatical but meaningless events. Another sample reads "Suddenly, a big hole in the ground. He picked it up and took it home." The first sentence has no verb, "he" was never introduced, and you can't pick up a hole. Earlier in the same sample the sky is "barking loudly". The text keeps the rhythm of a

children's story without the meaning. A character model this size learns spelling and local grammar well, but has very little sense of objects or cause and effect. A lower sampling temperature, or a bigger model, should reduce this; counting verbless sentences across 50 samples per setting would show whether it does.

Comparison with my teammate's model

Rajesh trained his own character-level GPT with different choices:

	Mine	Rajesh's
Width / heads / context	256 / 8 / 128 chars	192 / 6 / 192 chars
Parameters	4.83M, separate output layer	2.72M, output layer tied to embedding
Learning rate / weight decay	3e-4 / 0.01	1e-3 / 0.1
Data split	Random sample of 110K stories	Story-disjoint split with an end-of-story token
Hardware	Colab T4, fp16	Apple M4, fp32
Validation cross-entropy	0.639	0.711
Top-1 accuracy	79.55%	77.46%

My model reaches a lower validation loss, and the bigger model is probably most of the reason. The numbers aren't a perfectly fair comparison, though. His split is stricter, because no validation story ever appears in training, and his longer 192-character context should help with exactly the coherence problems described above. Both models show the same main weakness: greedy decoding loops, and the story loses track of its characters after a few sentences.

Task 2: Sentiment classification on Yelp Polarity

Data and exploration

I used the whole Yelp Polarity dataset. I split the official 560,000 training reviews 90/10, which gave 503,979 for training and 55,997 for validation. The official 38,000 test reviews were only touched for the final numbers.

The classes are exactly balanced (280,000 each), so accuracy isn't misleading here. Reviews average 133 words, with a median of 97 and a long tail out to 1,052 words. One thing I didn't expect was that negative reviews are noticeably longer than positive ones: 152 words on average against 115. Unhappy customers apparently have more to say.

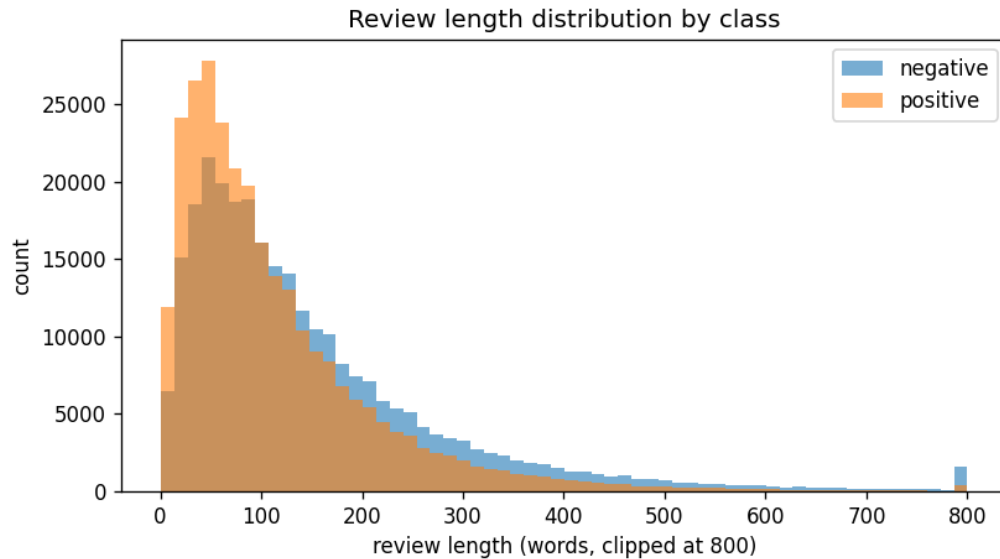


Figure 2. Review length distribution in the training set.

The dataset was mostly clean. I removed 24 training reviews that were empty or contained no letters at all. There were no exact duplicates, and I didn't remove anything from the test set.

Preprocessing

1. Lowercase the text, strip URLs and HTML, and expand contractions like "n't" to "not".
2. Remove punctuation and special characters.
3. Remove NLTK English stopwords, **except** negations and intensifiers such as not, no, never, but, too, very and only. The standard list would delete "not", which turns "not good" into "good".
4. Apply Porter stemming.
5. Split on whitespace, and build a vocabulary from the training split only: the 50,000 most frequent tokens that appear at least three times.

After cleaning, an average review is 72 tokens long. I cut sequences at 256 tokens, which only affects 2.3% of reviews, and just 0.4% of test tokens fall outside the vocabulary. Every model starts with a randomly initialised 128-dimensional embedding layer and learns it during training.

Models

I wanted three models that each see a bit more of the review's structure than the last one:

Model	Architecture	Hyperparameters	Parameters
Baseline	Average of word embeddings → hidden layer → output (fastText-style)	hidden 128, dropout 0.3, lr 2e-3	6,416,770
Experimental 1	CNN with filter widths 3, 4 and 5 (128 each) and max-over-time pooling (Kim, 2014)	dropout 0.5, lr 1e-3	6,597,762

Model	Architecture	Hyperparameters	Parameters
Experimental 2	One-layer bidirectional LSTM (128 per direction) with additive attention pooling	dropout 0.3, lr 1e-3	6,697,730

All three use Adam with batch size 128 and gradient clipping at 5, and train for up to 5 epochs with early stopping on validation macro-F1. They also share the same embedding size, so any difference between them comes from the architecture.

The baseline only knows which words are in a review, not what order they come in. It's cheap, it trains in about three minutes, and it gives an honest lower bound.

The CNN adds local word order. Its filters act like learned detectors for short phrases such as "not good" or "highly recommend", so I expected it to handle negation better.

The BiLSTM reads the whole review in both directions. The attention layer then decides which parts of the review matter most, instead of averaging everything equally. That should help with long reviews, where the actual verdict is often in a single sentence.

Results on the test set

Metric	Baseline	CNN	BiLSTM + attention
Accuracy	0.9332	0.9502	0.9581
Macro precision / recall / F1	0.9332 (all three)	0.9502 (all three)	0.9581 (all three)
ROC-AUC	0.9812	0.9883	0.9921
PR-AUC	0.9816	0.9882	0.9923
MCC	0.8665	0.9004	0.9161
Brier score (lower is better)	0.0503	0.0384	0.0323
Calibration error, ECE (lower is better)	0.0082	0.0105	0.0124
Accuracy 95% CI	0.931–0.936	0.948–0.952	0.956–0.960
Confusion [TN FP; FN TP]	[17739 1261; 1276 17724]	[18002 998; 895 18105]	[18196 804; 790 18210]
McNemar vs baseline (b / c)	—	688 / 1,332	542 / 1,485
Training time	173 s	791 s	≈10,600 s (logged 19,092 s)*
Inference speed	105,134 reviews/s	8,354 reviews/s	2,959 reviews/s
Peak memory (MPS)	2,810 MB	2,828 MB	4,006 MB

* While the BiLSTM was training, my laptop went to sleep for about 8,500 seconds, from 20:03 to 22:25 according to the system power log. The logged time includes that pause. I left the log as it is and report the corrected figure next to it.

Macro, micro and weighted precision, recall and F1 all come out identical. That's expected: the test set has exactly 19,000 reviews per class, and each model's errors are split almost evenly between the two.

The confidence intervals come from 1,000 bootstrap resamples. In McNemar's test, b counts reviews the baseline got right and the other model got wrong, and c counts the opposite. Both p-values are too small to print (below $1e-16$).

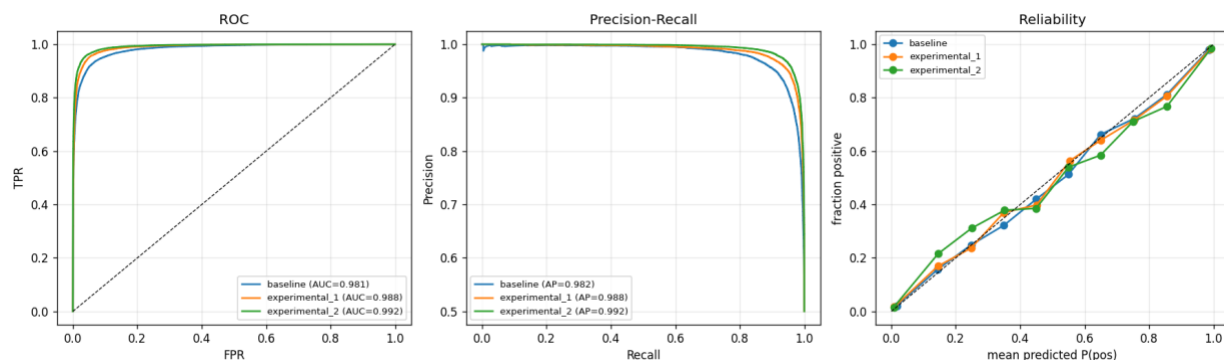


Figure 3. ROC, precision–recall and reliability curves on the test set.

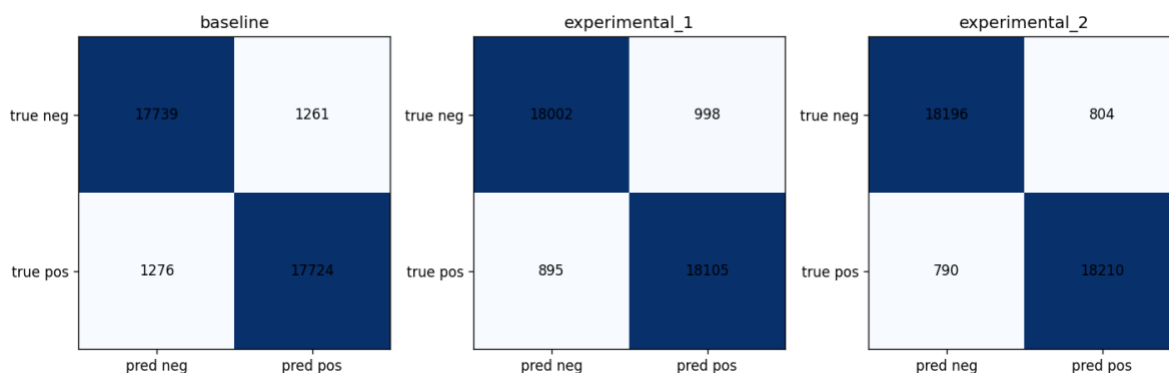


Figure 4. Test-set confusion matrices for the three models.

Where each model struggles

I also split the test set into slices that I expected to be hard, and checked each model's error rate on them:

Slice	Baseline	CNN	BiLSTM
Short reviews (bottom 25% by length)	6.68%	4.95%	4.41%
Long reviews (top 25% by length)	6.61%	5.16%	4.35%
Contains a negation	7.05%	4.99%	4.19%
Contains but / however / although	7.51%	5.44%	4.65%
Three or more exclamation marks	4.68%	3.34%	2.65%

The pattern is consistent. The baseline does worst on reviews with negation or a contrast word, which are exactly the cases where word order matters. Those are also the slices where the CNN and the BiLSTM gain the most: going from the baseline to the BiLSTM cuts the error rate on contrast reviews

from 7.5% to 4.65%. Reviews full of exclamation marks are the easiest for every model, my guess is that people who use three exclamation marks are rarely subtle about how they feel.

Comparing the three models

The BiLSTM is the best model on every accuracy measure and on every slice. Its confidence interval doesn't overlap with either of the others, and McNemar's test confirms both experimental models are genuinely better than the baseline, not just lucky on this test set.

The baseline does have one advantage: it's the best calibrated. Its predicted probabilities match how often it is actually right very closely (ECE 0.008). The BiLSTM is slightly over-confident; some of its wrong answers come with a probability above 0.999. Temperature scaling, fitted on the validation set, would be a cheap way to fix that without retraining.

The cost difference is large. The CNN takes about 4.5 times longer to train than the baseline, and the BiLSTM about 60 times longer. The BiLSTM is also 35 times slower at inference, because the recurrence has to run step by step and doesn't map well onto the Apple GPU. If I had to put one of these into production, I'd pick the CNN. It gets most of the BiLSTM's gain at a fraction of the cost.

Looking at the validation scores per epoch, the baseline levelled off after three epochs, and the CNN was still improving slightly at epoch 5. The BiLSTM peaked at epoch 4 (95.47% validation accuracy) and dipped very slightly at epoch 5 (95.41%), so more epochs probably wouldn't help it much.

Error analysis

I read through 20 of the BiLSTM's mistakes:

- the 5 most confident false positives;
- the 5 most confident false negatives;
- the 5 predictions closest to 50%;
- 5 random errors from the contrast slice.

Then I sorted them by what went wrong:

What went wrong	Count	Fix I would test
The verdict is on one side of a "but" and the model followed the other side	8	Mark the words after but/however during preprocessing, or weight the final clause more heavily in attention
Label noise: the star rating doesn't match the text	4	Hand-check a sample of confident errors; filter likely mislabeled training examples
The verdict comes after the 256-token cut	3	Keep the first and last 128 tokens instead of only the first 256
Past vs present: most of the review describes an earlier visit	2	Position-aware pooling that gives more weight to the end of the review
No real sentiment in the text (e.g. "where is this place?")	2	Let the model abstain when it's near 50% on very short reviews

What went wrong	Count	Fix I would test
Negated negatives ("too good to disrespect with a 2 star review")	1	Mark negation scope, e.g. turn "not bad" into a single not_bad token

Two things stood out. First, about a fifth of the confident mistakes weren't really mistakes. In one review someone wrote "Wow love the place and everything is very clean and new!" but gave it one or two stars. In another, a reviewer edited their text to complain but kept the old positive rating. Some of the remaining 4% error will never go away, because the labels come from star ratings and not from the text.

Second, contrast is the biggest real weakness. Reviews like "Food is good but the portions are small for what you are paying" give the model two signals pointing in opposite directions, and attention pooling tends to side with whichever part is longer. The BiLSTM already handles this better than the other two models, but it's still where most of the genuine errors come from.

Comparison with my teammate's models

Rajesh built three different models with a different pipeline:

	Mine	Rajesh's
Baseline	Averaged unigram embeddings → MLP: 93.32%	fastText with hashed bigrams: 95.15%
Experimental A	Multi-width CNN: 95.02%	BiGRU + attention: 95.23%
Experimental B	BiLSTM + attention: 95.81%	2-layer Transformer encoder: 93.23%
Model selection	Best validation macro-F1	Lowest validation log-loss
Truncation	First 256 tokens	First 160 + last 96 tokens

Comparing the two sets of results is useful. His bigram baseline is almost two points better than my unigram one, which shows how much of the value of word order is in two-word phrases like "not good". My recurrent model ended up about 0.6 points ahead of his. That could be because I trained it longer (5 epochs vs 3), or because I selected on F1 instead of loss, but I haven't tested either explanation.

His Transformer encoder, trained from scratch without any pretraining, was the weakest of his three. That fits what I'd expect at this data size, where the inductive bias of recurrence or convolution still helps. Both of us found that contrast reviews are the hardest large slice for every model.

Limitations and next steps

- Truncating at 256 tokens loses verdicts at the end of long reviews. Keeping the first and last part of the review, as Rajesh did, is an easy change.
- Stemming and stopword removal discard some nuance. A small ablation that keeps prices, capital letters and words like "just" would show how much this costs.
- Each model was trained once with one seed. The confidence intervals capture test-set noise, but not run-to-run variation.

- The BiLSTM is over-confident. Temperature scaling on the validation set should fix most of that.

Task 3: CycleGAN style transfer, photo ↔ Monet

Data

The competition gives 300 Monet paintings and 7,038 photos, all 256×256. I held out 1,000 random photos (seed 1337) for checkpoint selection and evaluation, and trained on the remaining 6,038 photos and all 300 paintings. The two domains are unpaired: every training step draws a random photo and an unrelated random painting. During training, images are resized to 286×286, randomly cropped back to 256×256 and randomly flipped.

Model

I used the standard CycleGAN layout: two generators (photo → Monet and Monet → photo) and two discriminators, one per domain.

- **Generators:** ResNet with 64 base filters and 9 residual blocks, InstanceNorm and reflection padding (11,378,179 parameters each). The decoder upsamples with nearest-neighbour resizing followed by a convolution, instead of a transposed convolution. My first run used transposed convolutions and produced visible checkerboard patterns, which this change removed.
- **Discriminators:** 70×70 PatchGAN with 64 base filters (2,764,737 parameters each).
- **Losses:** least-squares adversarial loss, cycle-consistency L1 with weight 10, identity L1 with weight 2.5, and a 50-image history buffer of fakes for each discriminator.
- **Small-data tricks:** with only 300 paintings, a discriminator memorises them quickly. DiffAugment applies a random translation to every image the discriminators see, real or fake, and both generators keep an exponential moving average (EMA) copy of their weights (decay 0.999).

In total the model has 28,285,832 parameters.

Training

The final model is the result of four runs. v1 (30K steps on a Colab T4) was the plain baseline. v2 (50K steps on a T4) switched to resize-convolution and added DiffAugment. v3 (12K steps on my laptop) tried a second short learning-rate cycle. The final run took v2's weights and trained them much longer on an RTX 4090 in the GPU lab:

Setting	Value	Why
Starting point	v2's weights (all four networks)	v2 already worked; the 4090's time went into refining it
Length	270 epochs × 1,000 steps = 270,000 steps, batch 1	In every earlier run, FID improved mostly while the learning rate decayed, so a long, slow decay was the best use of the time
Optimiser / LR	Adam, lr 2e-4, betas 0.5 / 0.999; constant for 108 epochs, then linear decay to 0	The standard CycleGAN schedule, stretched over the time budget

Setting	Value	Why
DiffAugment	translation only	Colour augmentation in v2 coincided with blob artifacts
Precision	bf16 autocast	Faster and lighter on memory than fp32; bf16 needs no loss scaling
Checkpoint selection	9 checks, both directions, raw or EMA weights for each generator	The score averages both directions; earlier runs only looked at photo → Monet

At each check, the run translated two separate sets of 300 held-out photos and the 300 paintings, and scored them with the instructor's FID/MiFID formula in both directions. It tried four weight combinations (raw or EMA for each generator) and kept the best. EMA weights won the first seven checks, by up to 1.8 points. By the end, though, the learning rate was close to zero and the raw weights had settled, so the final choice was epoch 269 with raw weights for both generators.

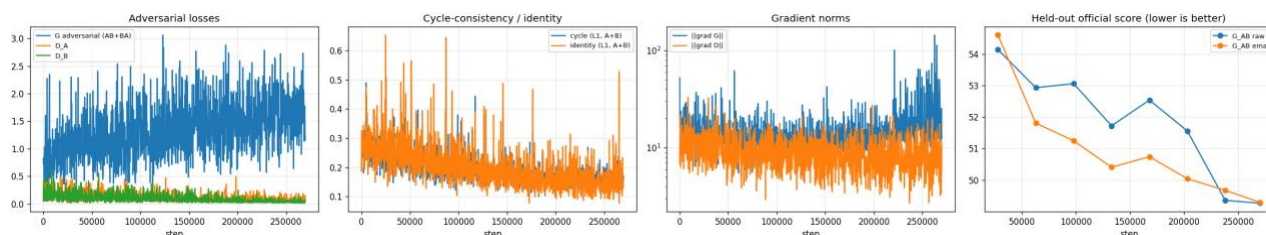


Figure 5. Adversarial losses, cycle and identity losses, gradient norms, and the held-out selection score over 270,000 steps.

Training was stable. There were no NaN or infinite losses in 270,000 steps, and no overflow steps were skipped. Gradients weren't clipped. The generator's gradient norm averaged 15.3 (95th percentile 27.8), with a few short spikes up to 520 late in the run, none of which caused a loss blow-up.

The cycle and identity losses fell steadily from about 0.25 to about 0.15. The discriminator losses stayed low (0.03–0.10), and the generator's adversarial loss drifted up from about 1.0 to 1.5 in the second half. That pattern is typical when the discriminators get stronger as the learning rate decays, and the selection score kept improving over the same period: 57.8 at the start, 51.8 at epoch 62, 50.0 at epoch 202 and 49.3 at epoch 269.

Results

Final model, instructor's script. I scored the final model with the instructor's evaluation notebook, unchanged except for the folder paths. It compares the first 300 translated images in each direction (sorted by name) with real images, and averages the two directions:

Direction	FID	MiFID
Photo → Monet (all 7,038 photos translated; first 300 scored)	96.956	0.4024
Monet → photo (all 300 paintings translated)	106.862	0.4139
Mean of both	101.909	0.4082

I uploaded this result on 4 October and it scored -51.1585 on the public leaderboard, which shows $-(\text{FID} + \text{MiFID})/2$.

Kaggle submission. After checking with the TA, the entry we report is my earlier v3 submission, which scored -45.9695 , the team's best; our team (PairProgramming_Team_06) is ranked 11th on the public leaderboard. Its submission.csv (1,91.537,0.402) comes from v3's G_AB applied to all 7,038 photos (images in outputs/history/v3/pred_B2A, weights in checkpoints/history/v3/). It was scored before the instructor's script was shared, with my own evaluate_competition_stats.py: photo $\rightarrow$ Monet only, the FID of all 7,038 translations against the competition's real_stats.npz. Re-running it on the committed images gives the same numbers. Because it uses one direction and far more images, its FID can't be compared with the 300-image numbers above.

Evaluation in both directions. These metrics use the 1,000 held-out photos and the 300 paintings, so the FIDs are lower than the instructor script's 300-image FIDs (FID is biased upward with fewer samples):

Metric	Photo $\rightarrow$ Monet	Monet $\rightarrow$ photo
FID	82.997	89.684
KID (mean $\pm$ std)	0.0126 $\pm$ 0.0008	0.0257 $\pm$ 0.0009
Precision / recall	0.576 / 0.630	0.697 / 0.428
Density / coverage	0.556 / 0.933	0.585 / 0.364
Cycle-reconstruction L1	0.1053	0.0841
LPIPS, input vs translation	0.4204	0.4019
Content cosine similarity, input vs translation	0.7316	0.7708
Identity L1	0.1003	0.0797

The two directions fail in opposite ways. Photo $\rightarrow$ Monet has high coverage (0.93) but lower precision (0.58): the translations spread across the whole range of Monet's work, but not every image is convincing. Monet $\rightarrow$ photo is the reverse: individual outputs look photographic (precision 0.70), but they only cover a narrow slice of real photos (coverage 0.36).

The cycle-consistency constraint works as intended. Translating there and back returns an image within about 0.08–0.11 mean absolute pixel difference of the original (on a 0–1 scale), and in the grids below the bottom row is almost identical to the top row.

Other costs: 24,770 s of training (6.9 h), 10.9 training steps per second, and 19,863 MB of peak GPU memory.



Figure 6. Photo $\rightarrow$ Monet on held-out photos. Top: input; middle: translation; bottom: cycle reconstruction.



Figure 7. Monet $\rightarrow$ photo. Top: painting; middle: translation; bottom: cycle reconstruction.

Human audit. Thirty held-out photos were chosen with a fixed seed before anyone looked at the outputs, and saved as anonymous photo | translation pairs. Two raters scored each pair independently from 1 to 5:

Criterion	Rater 1	Rater 2	Mean	Cohen's κ (quadratic)	Exact / within-1 agreement
Style (looks like a Monet)	3.57	3.73	3.65	0.13	33% / 83%
Content (scene preserved)	3.83	4.03	3.93	0.29	33% / 83%
Artifacts (5 = none)	3.43	4.07	3.75	0.28	27% / 83%
Overall			3.78	0.24	31% exact

Content was rated highest and style lowest, which fits the rest of the evaluation: the scene almost always survives, but the result doesn't always look like Monet. Agreement was only fair (κ 0.24). The raters were within one point 83% of the time, though, and rater 2 was more lenient, especially on artifacts. Most disagreements were a shift in scale, not opposite opinions.

Visual quality and failure cases

Most photo → Monet translations pick up a visible brush texture, broken-up foliage and Monet's softer palette. The recurring failures are:

- **Inventing colour on grey inputs.** A misty, almost colourless field (Figure 6, column 2) comes back tinted purple. The generator has learned that Monet paintings are colourful and adds colour even when the photo has none.
- **Losing contrast in dramatic skies.** The dark storm cloud (column 7) and the heavy sepia sky (column 3) come out pale and washed out.
- **Close-ups.** The pebbles (column 5) keep their shapes but get a speckled colour texture; brushstrokes sized for landscapes don't suit small objects.
- **Watermarks survive.** Photographer credits such as "© 2016 Thomas J. Astle" stay readable in both the translation and the reconstruction, because nothing in the loss removes text and the cycle loss rewards keeping it.

Monet → photo is the weaker direction. Most outputs are darker and muddier than the painting; Waterloo Bridge (Figure 7, column 8) becomes a near-black night scene. The still life of hanging game birds (column 4) has no equivalent among the outdoor photos, so the generator returns a darker copy of the painting. The poplars over water (column 7) break into streaks. One success: the pale, misty poplars in column 5 turn into a convincing blue sky with clouds.

Behind most of these is the same trade-off. A strong cycle loss keeps the content safe, but it also pulls each translation back toward its input, which limits how far the style can move.

What didn't work

UVCGAN-style generators. I implemented a U-Net generator with a small vision-transformer bottleneck, spectrally normalised discriminators and inpainting pre-training. Both pilot runs collapsed to the identity function: the outputs were almost the same as the inputs, and they scored about the same as untranslated images. The reconstruction-type losses outweighed the adversarial loss, and the pre-training started both generators off as copy networks.

PatchNCE continuation. Adding a contrastive PatchNCE loss to the final model made the images look more painterly, but the score got worse (FID 101.96 → 104.66 over 24K steps), so I stopped it. The restart learning rate (5e-5, about 40× the final one) probably knocked the model out of the spot it had settled into.

A Windows encoding bug. After training finished, the post-processing crashed because a summary file containing "λ" was written with Windows' default cp1252 encoding. Adding encoding="utf-8" fixed it, and the rerun finished in two minutes without retraining.

How the final design came about

Before the final run I trained three shorter versions of this model, and each one changed the design. The first used transposed convolutions and had checkerboard artifacts, so I switched to resize-convolution.

The second showed that FID improved almost only while the learning rate was decaying, which is why the final run uses a long schedule with a long decay. The third showed that the outputs needed more Monet-like texture, not better colour: matching the colour statistics exactly barely helped. Their configs, logs and samples are in the repository's history folders.

Comparison with my teammate's model

	Mine	Rajesh's
Generator	ResNet, 64 filters, 9 blocks, 11.4M params each	ResNet, 48 filters, 6 blocks, 4.4M params each
Discriminator	PatchGAN with InstanceNorm	PatchGAN with spectral norm
DiffAugment / identity weight	translation / 2.5	colour, translation, cutout / 5
Training	50K + 270K steps on 256×256 images, RTX 4090	24K steps on 128×128 crops, Apple M4
FID / MiFID (instructor's script)	101.909 / 0.4082	149.484 / 0.4135

Both models are scored with the same script, so this comparison is direct. Most of my advantage probably comes from the much longer training and the larger generator, which can afford more texture detail. Rajesh's spectral-norm discriminators gave him a very steady training balance with a model a third of the size. We both found Monet → photo to be the harder direction.

Limitations and next steps

- Monet → photo is about 10 FID behind photo → Monet. Leaving paintings with no photographic equivalent (still lifes, interiors) out of that direction's adversarial target, or giving them less weight, is the first thing I'd try.
- Continuing the finished model at a small learning rate (around 1e-6 to 5e-6) might still help; the PatchNCE run showed that restarting high does damage.
- Masking watermarks in the training photos would stop text from surviving into the outputs.
- The two raters agreed only fairly (κ 0.24). A short calibration round on a few shared examples before rating would make the human scores more reliable.

Reproducibility

Each task folder contains the code (`src/`), the config used for the run (`config.yaml`), the executed notebook, the final checkpoints, the metrics JSON files, the plots, and `results.md` / `failure_analysis.md`. The unedited raw training logs and the environment manifests are in `reproducibility/raw_logs/sivasurya_chandran/` and `reproducibility/manifests/`. The processed data isn't committed because of its size; `src/preprocess.py` rebuilds it from the raw datasets. For Task 3, the final weights, both prediction folders and the executed copy of the instructor's evaluation notebook are committed too.

	Task 1	Task 2	Task 3
Run from	task1_llm/sivasurya_chandran/	task2_sentiment/sivasurya_chandran/	task3_gan/sivasurya_chandran/
Commands	python src/preprocess.py --config config.yaml python src/train.py --config config.yaml python src/generate.py --config config.yaml	python src/preprocess.py --config config.yaml python src/train.py --config config.yaml python src/error_analysis.py --config config.yaml --model_name experimental_2	python src/run_final.py --hours 13 --variants V2_CONTINUE python evaluate_local.py --pred_dir outputs --out kaggle_submissions/final_submission.csv python evaluate_competition_stats.py --gen outputs/history/v3/pred_B2A --out submission.csv
Environment	Colab T4, Python 3.13, PyTorch 2.11 (CUDA 12.8)	Apple M5, macOS 26.6, Python 3.9, PyTorch 2.8	RTX 4090, Windows 11, Python 3.9, PyTorch 2.6 (CUDA 12.4)
Seed	1337	1337	1337

References

Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. ICLR.

Eldan, R., and Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? arXiv:2305.07759.

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. (2017). GANs trained by a two time-scale update rule converge to a local Nash equilibrium. NeurIPS.

Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8).

Joulin, A., Grave, E., Bojanowski, P., and Mikolov, T. (2017). Bag of tricks for efficient text classification. EACL.

Kim, Y. (2014). Convolutional neural networks for sentence classification. EMNLP.

Park, T., Efros, A. A., Zhang, R., and Zhu, J.-Y. (2020). Contrastive learning for unpaired image-to-image translation. ECCV.

Vaswani, A., et al. (2017). Attention is all you need. NeurIPS.

Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level convolutional networks for text classification. NeurIPS.

Zhao, S., Liu, Z., Lin, J., Zhu, J.-Y., and Han, S. (2020). Differentiable augmentation for data-efficient GAN training. NeurIPS.

Zhu, J.-Y., Park, T., Isola, P., and Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. ICCV.